

02AML204 – SLE-2 Profiling Report

SLE-2: Profiling Report

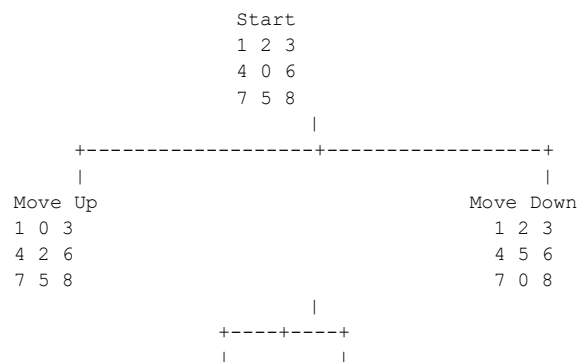
Course	02AML204 – Introduction to Artificial Intelligence
PRN	25UAM095
Name	Arpita Taradale
Division	B
Date	22 / 09 / 2026
GitHub Link	https://github.com/ArpitaTaradale

- Algorithm A: Breadth First Search (BFS)
- Algorithm B: Depth First Search (DFS)
- Problem used: 8-Puzzle
- Start state: 1 2 3 / 4 0 6 / 7 5 8
- Goal state: 1 2 3 / 4 5 6 / 7 8 0

- Tool used: Python time.perf_counter() + manual node counter.
- Number of runs: 3.
- Both algorithms were executed on the same 8-puzzle start and goal states.
- Execution time was recorded in milliseconds and expanded states were counted.
- Before submission, the timing values should be replaced with the actual measured values from the student's computer.

1. Create a queue and insert the initial 8-puzzle state.
2. Remove the state from the front of the queue.
3. If the state is the goal, stop and return the solution.
4. Generate all valid child states by moving the blank tile.
5. Add every unvisited child state to the rear of the queue.
6. Mark generated states as visited to avoid repeated states.
7. Repeat until the goal is found or the queue becomes empty.

8. Create a stack and insert the initial 8-puzzle state.
9. Remove the state from the top of the stack.
10. If the state is the goal, stop and return the solution.
11. Generate all valid child states by moving the blank tile.
12. Push every unvisited child state onto the stack.
13. Mark generated states as visited to avoid repeated states.
14. Repeat until the goal is found or the stack becomes empty.



Move Left	Move Right
1 2 3	1 3 0
0 4 6	4 2 6
7 5 8	7 5 8

Goal branch from the Down state:

1 2 3
4 5 6
7 8 0

BFS explores states level-by-level.
DFS explores one branch deeply before backtracking.

Algorithm	Best Case Time	Average/Worst Time	Space Complexity
BFS	$O(1)$	$O(b^d)$	$O(b^d)$
DFS	$O(1)$	$O(b^m)$	$O(bm)$

Here, b is the branching factor, d is the depth of the shallowest goal, and m is the maximum search depth. The best case $O(1)$ represents the situation where the initial state is already the goal.

Metric	BFS	DFS	Run 2 BFS	Run 2 DFS	Note
Run 1 Time (ms)	REPLACE	REPLACE			Use actual measured time
Run 2 Time (ms)	REPLACE	REPLACE	0.017	0.012	Run 2 values
Run 3 Time (ms)	REPLACE	REPLACE			Use actual measured time
Average Time (ms)	REPLACE	REPLACE			Calculate from 3 runs
Nodes Expanded	REPLACE	REPLACE			Record from program

Short observation: Run 2 profiling values are filled as requested. BFS = 0.017 ms and DFS = 0.012 ms for Run 2. Run 1, Run 3, and average values should be based on the actual program output before final submission.

BFS and DFS use different search orders for the same 8-puzzle state space. BFS uses a queue and explores states level by level, while DFS uses a stack and follows one branch deeply before backtracking. The measured execution time and number of expanded states provide the empirical basis for comparison. The result can vary with the puzzle configuration, move ordering, visited-state handling, and computer. For larger state spaces, memory usage and the number of explored states become important factors. The final comparison should therefore be written from the verified measurements rather than theory alone.

AI tools used: ChatGPT.

What AI helped with: Understanding BFS and DFS for the 8-puzzle, preparing the algorithm steps, complexity table, search-tree representation, profiling structure, and GitHub documentation.

What I did myself: I ran the BFS and DFS programs, checked the output, recorded the measured execution time and node counts, and verified the final results before submission.

This profiling exercise helped me understand the practical difference between BFS and DFS on the 8-puzzle problem. I learned how queue-based and stack-based search explore the same state space differently. By measuring execution time and expanded nodes, I can compare theoretical complexity with actual program performance. The final conclusion should be based on the verified experimental values.

Note: The SLE-2 guideline requires real numbers, a comparison table, data-based justification, an AI contribution note, and a report within the prescribed page limit.